Testing, Gas and Evaluation Report

Recorded 18 October 2026 · Zeran Yao (G2509029A)

1. Reproducibility and scope

Run python3 scripts/collect-evidence.py from the project root. Raw stdout/stderr, exit codes and UTC start times are stored in evidence/. Host: macOS-26.3.1-arm64-arm-64bit. The backend uses Python 3.12/FastAPI and pytest; frontend/contract tooling uses Node 24. The Python suite has 19 cases and the EVM suite has three tests; assertions are not counted as independent cases. This is functional evidence, not a security certification.

Check	Result	Exit code	Raw evidence
backend-tests	PASS	0	evidence/backend-tests.log
contract-tests	PASS	0	evidence/contract-tests.log
contract-compile	PASS	0	evidence/contract-compile.log
gas-benchmark	PASS	0	evidence/gas-benchmark.log
web-tests	PASS	0	evidence/web-tests.log
web-lint	PASS	0	evidence/web-lint.log
web-build	PASS	0	evidence/web-build.log

The Python runtime Docker image and combined-container Origin/session smoke are checked separately by scripts/docker-smoke.py. The browser workflow is checked separately by scripts/browser-smoke.py. Their raw evidence files disclose the exact local environment. No public Render or Sepolia result is claimed.

2. Tested scenarios and limitations

Backend cases cover actual Node-generated password compatibility; additive legacy migration; sessions and expiry; invitation escalation; wallet proof, expiry and replay; immutable/unique wallets; populated journal isolation; auditor read-only access and email redaction; pagination/filtering; CSV formula safety; precise wei analytics; decoding and rejecting untrusted receipts; Origin checks and request limits. Tests use temporary databases, real signatures and controlled RPC data.

Three EVM tests check funded accounting and invalid amounts; principal withdrawal with an empty reserve and retained interest; and top-up/partial-withdrawal accounting without retroactive interest. They use Ganache and simulated time. Malicious callbacks, failed recipients, competing claim fairness and arbitrary-sequence invariants are not covered.

Build/lint validate compilation, types and code rules. The separate Chromium/local-EVM workflow checks a rendered dashboard, three transaction previews/executions, confirmed journal imports, filters, chart data table, CSV and auditor controls, including a 390px layout. Its wallet is a test EIP-1193 adapter, not the MetaMask extension. No stakeholder feedback or actual user timing was gathered.

3. Controlled gas experiment

Compiler 0.8.30, EVM Shanghai, optimizer comparison off versus on (200 runs), fixed initial simulated chain time, fresh deployment per configuration, 500-basis-point rate. Sequence: deposit 1 ETH; owner funds 0.2 ETH; advance one year; claim; withdraw 1 ETH. All measurements are gas units, not ETH or milliseconds.

Method	Estimate off	Actual off	Estimate on	Actual on	Actual reduction
deposit	100864	95874	99727	94735	1.19%
fund	22852	22852	22724	22724	0.56%
claim	79709	54920	78748	53951	1.76%
withdraw	56393	41904	55195	40698	2.88%

Creation bytecode: 4812 bytes without optimizer and 2366 with it (50.83% smaller). Deployment gas: 1017375 versus 545131 (46.42% lower). Source: evidence/gas-benchmark.json. One run per configuration supports only a descriptive comparison.

Receipt gas can be below estimate because state changes, storage refunds and estimation strategy affect the result. Results apply to this particular state sequence. Repeated deposits and other network forks may differ. Public-network latency, current gas price and fiat cost are not measured. At a hypothetical 10 gwei gas price, an optimized claim of 53951 gas would cost 0.00053951 ETH. This assumption is illustrative, not a live quote. Ethereum explains the gas/price relationship in https://ethereum.org/developers/docs/gas/.

4. Measurable impact and validation

Observed: selected local accounting and authentication assertions pass, principal becomes zero after full withdrawal, and optimizer-enabled compilation reduces gas in this experiment. Not observed: improved financial access, lower user cost than existing services, higher trust, user satisfaction, public settlement time or scalability under concurrent load. Contextual research must not be reported as a result of this prototype.

Proposed pilot: recruit five consenting wallet-literate participants, ask each to link a wallet, deposit test ETH, identify reserve risk and locate a receipt, then record unaided completion, elapsed time, errors and feedback. Target: at least four complete unaided and can explain why a claim might fail. This pilot is a plan; no participant data or results are supplied.

5. Pending tests and release decision

Under the owner-authorised relaxed scope, one contract and four workflows are intentional. Auditor access, charting and fee previews are implemented. Next add hostile-recipient and disk restore tests, later-reorganisation reconciliation and load tests. Run real MetaMask/public-host checks and capture actual public receipts. Dependency advisories in development tools were reported during installation; no independent audit or vulnerability remediation assessment is claimed.

Release decision: local prototype checks support the agreed focused scope; original numeric thresholds are disclosed and enterprise production readiness is not established. See COMPLIANCE_MATRIX.md and DEPLOYMENT_RECORD.md for missing evidence.